

Implementación de conurrencia en módulos

**Problema adaptado:
Productor-Consumidor
(sensores de tráfico y módulos
de análisis)**

Sistemas operativos

**Institución Universitaria
Pascual Bravo**

Julian Quintero Copete

Lógica de la implementación

La simulación representa el flujo de datos del Sistema Integrado de Gestión de Tráfico (SIGET) con cuatro hilos que cumplen roles distintos, no repetitivos. Dos hilos productores generan datos con comportamientos diferentes: un sensor de flujo reporta con alta frecuencia el número de vehículos detectados, mientras que un sensor de incidentes (cámara) revisa con menor frecuencia, pero ocasionalmente reporta una severidad alta, simulando un accidente. Ambos depositan sus lecturas (etiquetadas por tipo) en una memoria compartida de tamaño fijo, organizada como una cola circular. Dos hilos consumidores retiran datos de esa misma cola, pero con propósitos distintos: el módulo de tendencia calcula un promedio móvil del flujo vehicular (tarea de cómputo ligero y continuo), mientras que el módulo de alertas identifica lecturas críticas (alto flujo o incidente severo) y simula el tiempo de despacho de una alerta real, incrementando un contador compartido de alertas emitidas. Esta separación de responsabilidades —una tarea estadística frente a una tarea de decisión/acción con mayor latencia— refleja cómo un sistema de tráfico real distribuye el trabajo entre monitoreo continuo y respuesta a eventos críticos.

Mecanismos de concurrencia y sincronización utilizados

El programa se implementó en C++ usando hilos y herramientas de sincronización que ofrece el sistema operativo (CreateThread, CreateSemaphore y CRITICAL_SECTION). Se usan dos semáforos contadores —clásicos del patrón productor-consumidor— que representan la cola compartida como un espacio limitado: uno cuenta los lugares vacíos (inicializado en la capacidad de la cola) y otro los datos ya disponibles (inicializado en cero). Se eligieron semáforos y no solo una sección crítica porque el problema no es únicamente de exclusión mutua, sino de coordinación por cantidad: un hilo debe poder esperar hasta que exista al menos una unidad de un recurso (espacio o dato), algo que una simple sección crítica no resuelve por sí sola. La escritura y lectura de la cola compartida, así como el contador de alertas, se protegen además con secciones críticas para exclusión mutua de grano fino, evitando condiciones de carrera al modificar variables compartidas desde varios hilos. El orden estricto "esperar recurso -> entrar a la sección crítica -> salir de la sección crítica -> avisar el recurso contrario" evita que un productor y un consumidor queden esperando mutuamente algo que el otro nunca liberará (interbloqueo). Un riesgo adicional que se consideró fue la inanición del módulo de tendencia si el módulo de alertas tomara siempre primero los datos disponibles; como ambos consumidores comparten el mismo aviso de "dato disponible", es el sistema operativo el que decide cuál despierta primero, por lo que en una versión futura convendría repartir la prioridad de forma más explícita si se notara que un consumidor queda rezagado frente al otro.

Observaciones de la ejecución

En la ejecución se aprecia que el sensor de flujo produce datos con mucha más frecuencia que el sensor de incidentes, y que el módulo de alertas introduce una pausa deliberada (400 ms) al procesar un dato crítico, simulando una tarea más costosa que el simple cálculo de promedio del módulo de tendencia. Pese a esta diferencia de velocidad entre los cuatro hilos, la cola compartida nunca se corrompe ni se bloquea de forma permanente: cuando se llena, los productores esperan; cuando está vacío, los consumidores esperan. El contador de alertas se incrementa de forma consistente y sin valores perdidos, lo que confirma que la sección crítica adicional sobre ese recurso compartido funciona correctamente.

Reflexión y conclusiones

Este ejercicio muestra por qué la concurrencia es indispensable en un sistema como el SIGET: los sensores generan información de forma continua e independiente de la velocidad a la que los módulos de análisis pueden procesarla, y distintos tipos de análisis (estadístico vs. de alerta) tienen costos de procesamiento muy diferentes. Una solución

secuencial obligaría a que el cálculo de tendencia espere a que termine el despacho de una alerta, o viceversa, degradando la capacidad de respuesta del sistema en tiempo real. Al mismo tiempo, el ejercicio deja claro que la concurrencia introduce riesgos que no existen en un programa secuencial: condiciones de carrera sobre la cola compartida y sobre el contador de alertas, y la posibilidad de interbloqueo si el orden de adquisición de recursos no es consistente entre productores y consumidores. Los semáforos contadores resultaron adecuados porque expresan de forma natural la disponibilidad de un recurso limitado (espacio o dato), y las secciones críticas complementan esa coordinación protegiendo el acceso de grano fino a la memoria compartida. En términos de gestión de recursos del sistema operativo, este mismo patrón —desacoplar productores y consumidores mediante una cola compartida y protegida— es el que usan los propios manejadores de dispositivos y colas de E/S del SO para evitar que un proceso rápido sature o corrompa el trabajo de uno más lento, y es exactamente el tipo de mecanismo que un sistema crítico de infraestructura urbana como el SIGET necesita para no colapsar ante cargas de datos desiguales.